

🔑 ¿QUÉ COLECCIÓN ELEGIR?

Característica	List	Set	Map
Orden garantizado	✓ Sí	· Dependee	· Dependee
Duplicados	✓ Sí	✗ No	Claves no
Acceso por índice	✓ Sí	✗ No	✗ No
Extiende Collection	✓ Sí	✓ Sí	✗ No
Estructura	Elementos	Únicos	Clave-Valor

🔑 CUÁNDO USAR CADA IMPLEMENTACIÓN

Clase	Cuándo usarla
ArrayList	Acceso frecuente por índice · pocas inserciones en medio
LinkedList	Muchas inserciones/eliminaciones · usar como pila o cola
HashSet	Eliminar duplicados · comprobación rápida de existencia
LinkedHashSet	Como HashSet pero manteniendo el orden de inserción
TreeSet	Set ordenado automáticamente (requiere Comparable)
HashMap	Diccionario clave-valor · acceso rápido sin orden
LinkedHashMap	HashMap con orden de inserción conservado
TreeMap	Map con claves ordenadas automáticamente

✗ ERRORES MÁS COMUNES

- No usar genéricos (*raw types*) → `ClassCastException` en runtime
- Intentar `get(int)` en un `Set` → `Set` no tiene acceso por índice
- No sobrescribir `equals()` / `hashCode()` → `HashSet`/`HashMap` no detecta duplicados
- Modificar colección dentro de `for-each` → `ConcurrentModificationException`
→ usa siempre `Iterator` con `it.remove()`
- Confundir `remove(int)` vs `remove(Object)` en `ArrayList`

🔑 ORDENACIÓN – CLASE COLLECTIONS

```
import java.util.Collections;
Collections.sort(lista);           // orden natural (Comparable)
Collections.sort(lista, comp);     // orden con Comparator
Collections.reverse(lista);        // invertir
Collections.shuffle(lista);        // aleatorio
Collections.max(lista);            // mayor elemento
Collections.min(lista);            // menor elemento
```

🔑 COMPARABLE VS COMPARATOR

COMPARABLE – ORDEN NATURAL

COMPARATOR – ORDEN EXTERNO

```
class Persona
implements Comparable<Persona> {
    String nombre; int edad;
    @Override
    public int compareTo(Persona o){
        return Integer.compare(
            this.edad, o.edad);
    }
}
```

```
Comparator<Persona> c =
    (Persona a, Persona b) →
        a.nombre
        .compareTo(b.nombre);

Collections.sort(
    lista, c);
```

Dentro de la clase · usado por `Collections.sort()`, `TreeSet`, `TreeMap`

Externo a la clase · lambda · permite múltiples criterios

🔑 PATRONES DE RECORRIDO – MAP

POR KEYSET()

```
for (String k : mapa.keySet())
    System.out.println(k + " → " + mapa.get(k));
```

POR ENTRYSET() – MÁS EFICIENTE

```
for (Map.Entry<String,Integer> e
    : mapa.entrySet()) {
    System.out.println(
        e.getKey() + " → " + e.getValue());
}
```

ELIMINAR DURANTE RECORRIDO DE MAP

```
Iterator<Map.Entry<String,Integer>> it
    = mapa.entrySet().iterator();
while (it.hasNext()) {
    Map.Entry<String,Integer> e = it.next();
    if (e.getValue() ≤ 18) it.remove();
}
```

🌟 EJEMPLO INTEGRADOR – CLASE PRODUCTO

```
class Producto {
    private String nombre; private double precio;
    public Producto(String n, double p){ nombre=n; precio=p; }
    public String getNombre(){ return nombre; }
    public double getPrecio(){ return precio; }
    @Override public String toString(){
        return nombre + " - " + precio + "€"; }
}

List<Producto> prods = new ArrayList<>();
prods.add(new Producto("Teclado", 30));
for (Producto p : prods) System.out.println(p);
```

⚠ Sin `toString()`, `System.out.println(obj)` imprime la dirección de memoria.

DAM · PROGRAMACIÓN · UNIDAD 10

Colecciones de Datos

JAVA · CHULETARIO · TRÍPTICO A4

CONTENIDOS DEL TRÍPTICO

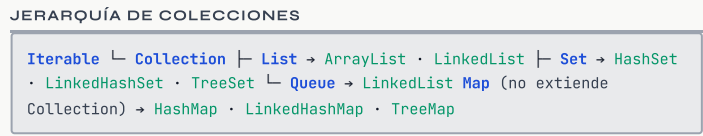
Interior izq. — List, ArrayList, Iterator, ListIterator

Interior cent. — LinkedList (lista/cola/pila), Set

Interior der. — Map, HashMap, métodos, recorridos

Solapa — Comparable vs Comparator, patrones Map, clase integradora

Contraportada — Tabla comparativa, cuándo usar cada clase, errores comunes



ARRAYS VS COLECCIONES

Arrays ✗ Tamaño fijo · No crecen · Métodos limitados · Sin algoritmos	Collections ✓ Dinámicas · Métodos útiles · Algoritmos integrados · Trabajan con objetos
---	---

CARA A → Exterior (portada, solapa, contraportada)
CARA B → Interior (abre el tríptico para ver la teoría)
Imprimir: A4 horizontal · doble cara · voltear por lado largo

 **INTERFAZ LIST – TEORÍA**

- Mantiene el **orden de inserción**
- Permite **elementos duplicados**
- Acceso por **índice** con `get(i)`

MÉTODOS DE LIST / ARRAYLIST

<code>add(e)</code> añade al final	<code>add(i, e)</code> inserta en posición i
<code>get(i)</code> elemento en posición i	<code>set(i, e)</code> reemplaza posición i
<code>remove(i)</code> elimina por índice	<code>remove(obj)</code> elimina 1ª ocurrencia
<code>contains(e)</code> ¿existe el elemento?	<code>size()</code> número de elementos
<code>isEmpty()</code> ¿está vacío?	<code>clear()</code> vacía la lista
<code>indexOf(e)</code> primer índice de e	<code>subList(a, b)</code> sublista [a,b)
<code>addAll(col)</code> añade colección entera	<code>toArray()</code> convierte a array

 ITERATOR Y LISTITERATOR

ITERATOR

<code>hasNext()</code> ¿hay más elementos?	<code>next()</code> devuelve el siguiente
<code>remove()</code> elimina el actual	

LISTITERATOR – MÉTODOS ADICIONALES

<code>hasPrevious()</code> hay elemento anterior	<code>previous()</code> devuelve el anterior
<code>nextIndex()</code> índice del próximo	<code>previousIndex()</code> índice del anterior
<code>set(e)</code> reemplaza el actual	<code>add(e)</code> inserta en posición actual

 **LINKEDLIST – TEORÍA**

✓ Inserciones/eliminaciones rápidas

✗ Acceso por índice más lento que ArrayList

MÉTODOS DE LISTA / DEQUE

<code>addFirst(e)</code> añade al inicio	<code>addLast(e)</code> añade al final
<code>getFirst()</code> primer elemento	<code>getLast()</code> último elemento
<code>removeFirst()</code> elimina el primero	<code>removeLast()</code> elimina el último
<code>indexOf(e)</code> primera posición de e	<code>lastIndexOf(e)</code> última posición de e
<code>descendingIterator()</code> iterador inverso	

COLA – QUEUE

<code>offer(e)</code> añade al final	<code>offerFirst(e)</code> añade al inicio
<code>peek()</code> ver primero sin eliminar	<code>poll()</code> obtiene y elimina primero
<code>pollFirst()</code> elimina el primero	<code>pollLast()</code> elimina el último

PILA – STACK

<code>push(e)</code> añade al inicio (tope)	<code>pop()</code> elimina y devuelve el tope
--	--

 SET – TEORÍA

- NO permite **elementos duplicados**
- Sin acceso por índice
- Orden depende de la implementación

MÉTODOS DE SET

<code>add(e)</code> añade si no existe	<code>remove(e)</code> elimina el elemento
<code>contains(e)</code> ¿existe?	<code>size()</code> número de elementos
<code>addAll(col)</code> unión	<code>removeAll(col)</code> diferencia
<code>retainAll(col)</code> intersección	<code>containsAll(col)</code> ¿contiene todo?
<code>iterator()</code> devuelve iterador	<code>forEach(λ)</code> recorre con lambda

 **MAP – TEORÍA**

- Pares **clave única** → **valor**
- Valores pueden repetirse
- **No hereda** de Collection

MÉTODOS DE MAP

<code>put(k, v)</code> añade o reemplaza	<code>get(k)</code> obtiene valor por clave
<code>remove(k)</code> elimina por clave	<code>containsKey(k)</code> ¿existe la clave?
<code>containsValue(v)</code> ¿existe el valor?	<code>size()</code> número de pares
<code>isEmpty()</code> ¿está vacío?	<code>clear()</code> vacía el mapa
<code>keySet()</code> Set con todas las claves	<code>values()</code> colección de valores
<code>entrySet()</code> Set de pares Entry K-V	<code>putIfAbsent(k, v)</code> añade si no existe
<code>replace(k, v)</code> reemplaza valor	<code>getOrDefault(k, d)</code> valor o default
<code>forEach(λ)</code> recorre pares K-V	

EJEMPLO – HASHMAP

```
Map<String,Integer> edades = new HashMap<>();
edades.put("Ana",25); edades.put("Luis",30);
System.out.println(edades.get("Ana")); // 25
for(Map.Entry<String,Integer> e : edades.entrySet())
    System.out.println(e.getKey()+"-"+e.getValue());
```

⚙ GENÉRICOS – SEGURIDAD DE TIPOS

```
// Sin genérico ⚠ raw type – peligroso
List lista = new ArrayList();
lista.add("hola"); lista.add(42); // mezcla tipos
String s = (String) lista.get(1); // ClassCastException!

// Con genérico ✓ seguro
List<String> lista = new ArrayList<>();
lista.add(42); // error de compilación ✓
String s = lista.get(0); // sin cast necesario
```

El operador diamante `<>` (Java 7+) infiere el tipo automáticamente: `new ArrayList<>()` en lugar de `new ArrayList<String>()`